The Essential of Chapter 4

Query processing in four pages: pipeline breakers, the passes of an external sort, the cost of each join, `Batches`, which join wins

Advanced Data Management

2026-10-31

Notation: B pages of a file, B_R, B_S pages of the join inputs, $|R|$ *rows* of R, M buffer pages an operator may use (`work_mem`, 8 KB per page; default 4 MB = 512 pages), h tree height. `customers` 10,000 rows / 73 pages, `orders` 500,000 rows / 3,185 pages.

1 Iterators and pipeline breakers

Concept: Streaming and blocking

Operators pull tuples with `next()`; a **pipelined** operator passes a tuple on as soon as it sees it (scan, filter, projection, probe side of a join). A **pipeline breaker** consumes all of its input before emitting any: **Sort**, the **Hash build** of a hash join, **HashAggregate** (a SUM per group is final only after the last row). A breaker materialises its input in `work_mem` or spills to temp files: that is where a plan's I/O concentrates. In `EXPLAIN ANALYZE`, `actual time=a..b`: a breaker shows $a \approx b$, a streaming node a wide gap.

Mock v2's December report, bottom-up: `Seq Scan customers` (streams, 73 pages) → `Hash` (blocking: 10,000 rows, 519 kB, `Batches: 1`) → `Hash Join` (streams the probe side, `orders`, once the table exists) ← `Seq Scan orders` (streams; 42,299 kept, 457,701 removed by the filter) → `HashAggregate` by country (blocking) → `Sort` revenue DESC (blocking, 5 rows). Two of five stream; the first row appears only at the end.

Mock drill

Mock v2 C1: the five nodes, each labelled blocking or streaming with one reason. **Quiz A1–A2, B4**: name three breakers; the aggregate that is *not* one is GroupAggregate over sorted input (§6).

2 External merge sort

Mechanism: Passes and cost

Pass 0: read M pages, sort in memory, write a sorted **run**: $\lceil B/M \rceil$ runs. **Merge passes:** one buffer for output, $M-1$ for inputs, so each pass merges $M-1$ runs into one; repeat until one run. Every pass reads and writes the file: $2B$.

$$\text{runs} = \lceil B/M \rceil, \qquad \text{fan-in} = M-1,$$
$$\text{merge passes} = \lceil \log_{M-1} \text{runs} \rceil, \qquad \text{I/O} = 2B\,(1 + \text{merge passes}).$$

Never compute the log: divide the run count by $M-1$, rounding up, until 1, and count. **Two passes** (one merge) when $\lceil B/M \rceil \le M-1$, i.e. $M \approx \sqrt{B}$. **One pass** when $M \ge B$ (read once, write nothing).

Key insight. Passes are a ceiling of a logarithm: memory buys whole passes, each worth $2B$. Doubling M removes at most one pass; nothing below $M \ge B$ removes the last merge pass.

Question	B	M	runs	fan-in	passes	I/O = $2B$×passes
Mock 1, 1.1	800	21	39	20: $39 \to 2 \to 1$	**3**	4,800
Mock v2 A5	6,000	100	60	$99 \ge 60$	**2**	24,000
Mock v2 B1	1,200	25	48	24: $48 \to 2 \to 1$	**3**	7,200
Mock v2 B1	1,200	50	24	$49 \ge 24$	**2**	4,800: a third less, not half
Mock 1, 3.4(a)	1,000,000	8,000	125	$7{,}999 \ge 125$	**2**	4,000,000 ($\approx$ 32 GB)

Traps: the run count is not the pass count (39, 60); one pass needs $M \ge B$ (1,200 pages = 9.6 MB for B1). In PostgreSQL: `Sort Method: quicksort Memory:` fits, `external merge Disk:` spilled; `work_mem` is *per sort or hash, per connection*, so raise it with `SET LOCAL` for one statement, never globally.

Mock drill

Mock 1, 1.1 / Mock v2 A5: letter plus "runs = $\lceil B/M \rceil$, fan-in $M-1$, divide until 1, add pass 0", name the trap. **Mock v2 B1**: the two computations with units, then "a third, not a half; single pass needs 9.6 MB". **Mock 1, 3.4(a)**: runs, passes, I/O, the $\sqrt{B}$ remark.

3 Nested-loop joins

Mechanism: Three variants

Simple: for each *row* of R scan S: $B_R + |R| \cdot B_S$ (absurd for real tables). **Block:** read R in chunks of $M-2$ pages (one buffer for the current S page, one for output), scan S once per chunk: $B_R + \lceil B_R/(M-2) \rceil \cdot B_S$. Smaller input outside; when $M - 2 \ge B_R$ it is $B_R + B_S$. **Index:** for each row of R descend a B^+ tree on S's key: $B_R + |R|\,(h + \text{matches})$; in a plan, `Nested Loop` over an inner `Index Scan` with `loops=`$|R|$. Unbeatable for a tiny outer (`c.id = 42`: $1 + (3+1)$ pages against 3,185), a disaster for a large one (`loops=20000`: 641 ms against 232 for the hash join). All three

accept *any* condition (`<`, `BETWEEN`).

Mock 1, 1.5: $B_R = 300$, $B_S = 3{,}000$, $M = 32$: chunks $\lceil 300/30 \rceil = 10$; $300 + 10 \times 3{,}000 =$ **30,300**. Traps: 9,300 (chunks of 100, needs $M = 102$); 900,300 (simple loop with 300 rows); 3,300 ($B_R + B_S$, needs $M \geq 302$ or a hash join).
Mock v2 B3(a)(c): $B_R = 80$, $B_S = 3{,}200$. $M = 10$: $\lceil 80/8 \rceil = 10$ chunks, $80 + 32{,}000 =$ **32,080** (S read ten times). $M = 100$: one chunk, **3,280** (S read once).

Mock drill

Mock 1, 1.5: letter, chunk computation ($M - 2$, rounded up), sum, the three traps named. **Mock v2 B3(a)(c)**: both memories, why $M = 100$ reads S once. **Mock v2 D2**: the `c.id = 42` lookup and "the only algorithm for non-equi joins".

4 Sort-merge join and hash join

Mechanism: Sort-merge

Sort both on the key (§2), then merge with one pointer each: $\text{sort}(R) + \text{sort}(S) + (B_R + B_S)$. Wins when the sorts are free (inputs arrive sorted from index scans, or the query wants `ORDER BY`/`GROUP BY` on the key anyway); handles range joins; output is sorted. Loses when nothing is sorted and memory is short (class example: 432 ms against 232 for hash, first row at 247 ms).

Mechanism: Hash join: build, probe, Grace

Build: read the *smaller* input R once into an in-memory hash table (the breaker). **Probe**: read S once, look up each row, emit matches (streaming, output in S's order). Fits in `work_mem`: $\mathbf{B_R + B_S}$, the minimum any join can cost. Does not fit: **Grace**: partition both inputs with the same h_1 to disk (read and write, $2(B_R + B_S)$), then join partition pairs ($B_R + B_S$): $\approx \mathbf{3(B_R + B_S)}$. Equality only. In a plan: `Hash ... Buckets: 16384 Batches: 1 Memory Usage: 519kB`. `Batches: 1` = the build fit; `Batches: 8` = both inputs split into 8 partitions, 7 written to temp files (`temp written=` on the Hash node; `temp read/written` at the top for the probe rows of batches 2–8). These are a breaker spilling, not the buffer pool.

Mock v2 A4: build on `customers` (73 pages, 519 KB, fits in 4 MB); `orders` (25 MB) would batch. "Either, same cost" holds only when both fit.
Mock v2 B3(b): $B_R = 80 > M = 10$: Grace, $3 \times 3{,}280 =$ **9,840**, 3.3× cheaper than the block loop's 32,080. **B3(c)**, $M = 100$: fits, 3,280; the block loop also reads S once at this memory, so **the two tie**.
Mock 1, 3.4(b): `users` 30,000 pages, `events` 1,000,000, `work_mem` 8,000

pages. Build on users; 30,000 > 8,000: Grace, ≈ 4 batches, 3 × 1,030,000 = **3,090,000**. Sort-merge 4,000,000 + 120,000 + 1,030,000 = 5,150,000; block loop $30{,}000 + 4 \times 10^6 = 4{,}030{,}000$. Hash wins; a filter shrinking users to 3,000 pages makes the build fit: ≈ 1,030,000.

Mock v2 C5: `work_mem = '64kB'` < 519 kB: `Batches: 8, Memory Usage: 80kB`, `temp written=28` (7 spilled build partitions), `temp read=183 written=183` (probe rows of batches 2–8); 62.3 → 71.5 ms (+15%, tiny temp files served by the OS cache). Fix: `work_mem` above the build's need with margin (≥ 1 MB; the 4 MB default is right), set per statement with `SET LOCAL`.

Mock drill

Mock v2 A4: customers, "$B_R + B_S$ holds only while the build fits; 73 pages fit, 3,185 would batch". **B3(b)(c)**: does R fit in M, which variant, $3(B_R + B_S)$ or $B_R + B_S$, the ratio, the tie. **Mock 1, 3.4(b)**: join, build side, fits?, batches, I/O, two alternatives, the plan line. **Mock v2 C5**: what changed in the Hash node, the two temp lines, `SET LOCAL`.

5 Which join wins, and when each is a disaster

Algorithm	Cost (pages)	Clearly best	Disaster
Block nested loop	$B_R + \lceil B_R/(M-2) \rceil B_S$	one input fits in a chunk; any condition	two large inputs, no index: 30,300 against 3,300
Index nested loop	$B_R + \lvert R \rvert (h + \text{matches})$	tiny outer, index on the inner key: `c.id = 42`	large outer: 20,000 descents, 641 ms against 232
Sort-merge	sort(R) + sort(S) + $B_R + B_S$	inputs already sorted or sorted output wanted; ranges	neither sorted, little memory: two external sorts first
Hash	$B_R + B_S$ fits; $3(B_R + B_S)$ Grace	large equi-join, no useful order, build fits	non-equality; build ≫ memory (`Batches` ≫ 1); skewed key

The winner depends on the condition (equality or not), sizes (is one side tiny?), indexes on the inner key, order present or required, and memory for a build or sort; each algorithm is the best choice in another's disaster, so the optimiser keeps all three. `SET enable_hashjoin = off` is a 10^{10} cost *penalty*, not a prohibition: the planner falls back to merge, then nested loop, and runs the penalised plan if nothing else exists. One-query experiments only, then `RESET ALL`.

Mock v2 D2 (`enable_nestloop` and `enable_mergejoin` off database-wide): *claim* hash join is not always best; *mechanism* nested loop for a tiny indexed outer (1 + (3 + 1) pages against 3,185) and for non-equi joins; merge join when both inputs arrive sorted ($B_R + B_S$, streaming, free `ORDER BY`); *consequence* every point query becomes a whole-table hash join (1 ms → 60 ms), sorted outputs re-sorted, hash tables compete for `work_mem`. Reset both; fix the bad estimate behind the one helped report (`ANALYZE`, `CREATE STATISTICS`).

Mock drill

Mock 1, 2.4: the table as prose, one best case and one disaster per algorithm with a number, then "why keep all three". **Mock v2 D2**: claim / mechanism / number / consequence. **Mock v2 B3(d)**: switching hash joins off tripled the report (32,080 against 9,840) for every join in the session; memory, not the algorithm, fixed it (tie at $M = 100$); re-enable, keep the larger `work_mem`; a planner setting is not a fix for a correctness bug.

6 Aggregation: hash or sort

HashAggregate: one entry per group, updated as rows stream in, emits at the end (breaker); chosen when groups are few (`GROUP BY country`, 12 groups, 24 KB, 6 ms). **GroupAggregate**: walks input sorted on the key, a group is complete when the key changes, streams; chosen when groups are many (`GROUP BY name`, 19,104 groups: `Sort` then `GroupAggregate`, 26 ms) or the input already arrives ordered (index scan). `DISTINCT` is the same choice.

Mock drill

Mock v2 C1: HashAggregate is blocking because no country total is final before the last row. **Quiz A18, B8, C4**: hash when few groups, sort + group when many or ordered input; only the latter streams.

Warning / Gotcha: The four mistakes that cost points

- $|R|$ where B_R belongs: simple loop uses rows, block loop uses pages in chunks of $M - 2$ (the 900,300 and 9,300 distractors).
- Fan-in M, forgetting pass 0, counting reads only: fan-in $M - 1$, passes $1 + \lceil \log_{M-1} \text{runs} \rceil$, $2B$ each; the run count is never the pass count.
- "Doubling memory halves the sort." At most one pass removed; the last merge pass stays until $M \geq B$.
- Hash join for everything: equality only; $B_R + B_S$ only while the build fits, else $3(B_R + B_S)$ with `Batches` > 1; a tiny indexed outer makes the nested loop hundreds of times cheaper.

Summary: The essential in five lines

1. Tuples stream except through breakers (Sort, Hash build, HashAggregate), which materialise and may spill.
2. Sort: runs $\lceil B/M \rceil$, fan-in $M - 1$, merge passes $\lceil \log_{M-1} \text{runs} \rceil$, I/O $2B(1 +$ merge passes); two passes at $M \approx \sqrt{B}$, one at $M \geq B$.
3. Block loop $B_R + \lceil B_R/(M-2) \rceil B_S$, small input outside; index loop $B_R + |R|(h +$ matches); both take any condition.
4. Hash: build on the smaller side, $B_R + B_S$ if it fits, else Grace $3(B_R + B_S)$, `Batches` > 1. Sort-merge: two sorts $+B_R + B_S$, free when sorted, handles ranges.
5. No join dominates: condition, sizes, indexes, order, memory; `enable_*` are penalties. Aggregation mirrors joins: hash for few groups, sort for many.